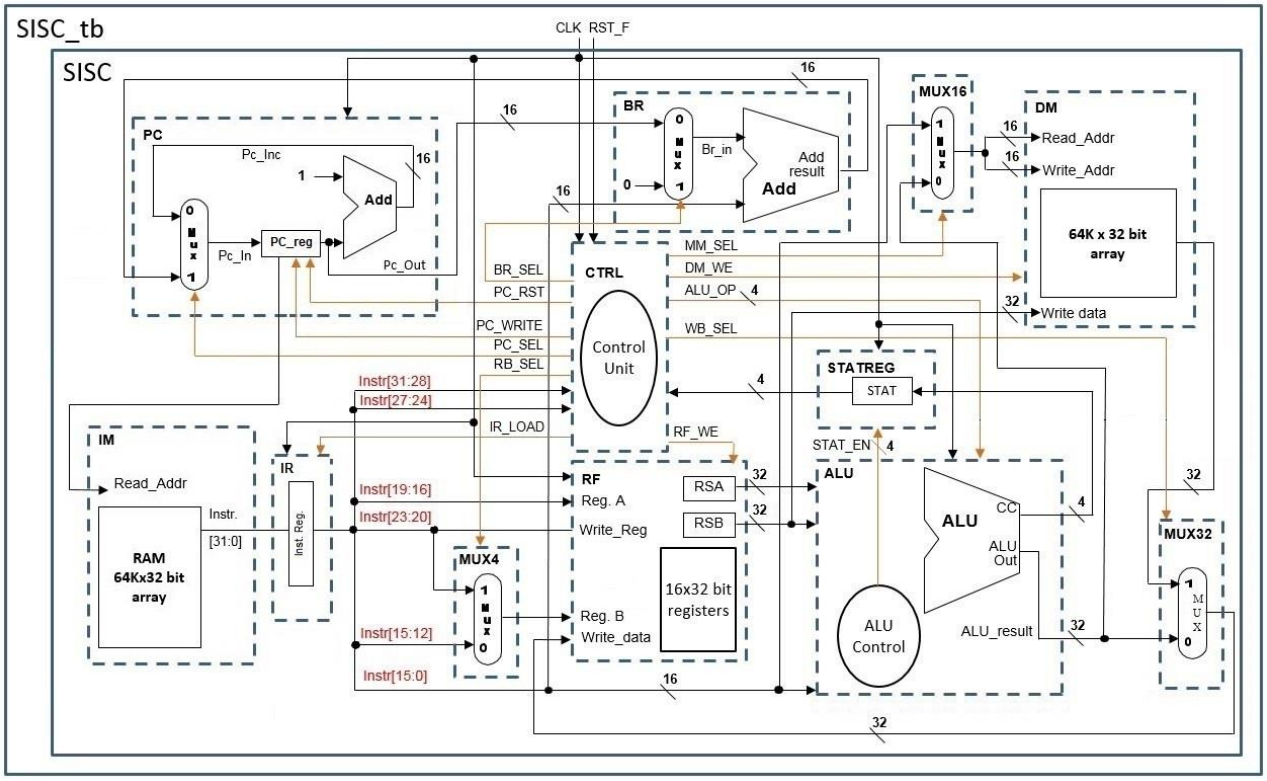
ECE:3350 Spring 2025 - Computer Architecture and Organization
Simple Instruction Set Computer (SISC) Project

Part 3: Due Monday, May 5, @ 8:30 am

This part has two distinct goals: **1)** Complete the datapath by adding the data memory and modifying the control unit to support LDX, LDA, STX, and STA load and store instructions (as shown below), and **2)** Write two machine-language programs to test your design. More details are given below.



You are given:

- Your own solution to Part 2. **Again, you will not be able to succeed in Part 3 without first completing Part 2.**
- An imem.data file containing a program designed to test all of the instructions from Parts 1 and 2 and the LDX, LDA, STX and STA instructions of part 3. This is so that you can be sure your modifications for Part 3 have not broken your previous functionality.
- Three example data memory files (dmem.data, sort_data.data and mult_data.data).
- The Verilog files dm.v, mux4.v and mux16.v (complete with descriptions), which you are not to modify.

- The diagram shown above.

You are required to:

- Modify the ctrl and sisc modules to include the new control lines and modules for the load and store instructions. Note that your sisc_tb file should not need to change from Part 2.
- Simulate your design using the supplied imem.data file and verify that the load and store instructions are working

- Write two machine language programs:
  - Sort a list of N signed, 32-bit integers stored in main memory using the bubble sort algorithm. The number of integers in the list, N, will be stored in memory location 0, with the integers stored at addresses 1 through N. After execution of your program, the integers should be in ascending order (least integer stored at address 1, and greatest stored at address N). An example list is provided in sort_data.data. **Save this instruction memory file as sort_instr.data.**
  - Multiply two unsigned, 32-bit integers stored in memory and write the resulting 64-bit product back to memory. The two integers will be stored in memory locations 0 and 1, and the product should be written to locations 2 and 3, such that the most significant bits are at address 3. An example data file is provided as mult_data.data. **Save this instruction memory file as mult_instr.data.**
- **NOTE!** In order to run your sort program or multiply program, you must modify the im.v and dm.v files to load the correct instructions and data.

Details/Notes:

- Note that the STX instruction requires two outputs from the register file; the Rs register needed to compute the memory address and the Rd register value to be written to memory. The mux4 is connected to the register B select input of the register file to do this. The RB_SEL control signal controls the mux4.

Submission Overview:

- Compress your project folder, along with the 'work' directory, the three instruction memory files, the data memory files, and all .v files into a .zip file named "Part3.zip".

Rubric:

Load and Store Implementation	25 pts
Swap Implementation	25 pts
Bubble Sort	20 pts
Multiplication	20 pts
AllInstr.data	10 pts
Total	**100 pts**